CNN Transfer Learning

Structured Notes

Why Training from Scratch is Hard,

Transfer Learning Strategies,
Feature Extraction, Fine-Tuning, and PyTorch Implementation

Contents

1 Why Training a CNN from Scratch is Hard **2**
1.1 The Parameter Challenge . . . 2
1.2 Data Requirement Problem . . . 2
1.3 Computational Cost . . . 2
1.4 The Hidden Issue: Feature Learning Complexity . . . 2

2 What is Transfer Learning? **3**
2.1 Core Idea . . . 3
2.2 Why Transfer Learning Works . . . 3
2.3 When Transfer Learning May Fail . . . 3

3 ImageNet and Pre-trained Models **4**
3.1 The ImageNet Dataset . . . 4
3.2 Famous Pre-trained CNN Architectures . . . 4

4 CNN Feature Hierarchy **4**

5 Two Strategies in Transfer Learning **5**
5.1 Strategy 1: Feature Extraction . . . 5
5.2 Strategy 2: Fine-Tuning . . . 5
5.3 Comparison: Feature Extraction vs. Fine-Tuning . . . 6

6 Freezing Layers in PyTorch **6**

7 Practical Transfer Learning Workflow **6**
7.1 Step-by-Step Pipeline . . . 6
7.2 Loading Models: With and Without Pre-trained Weights . . . 7
7.3 Inspecting the Final Layer . . . 7
7.4 Modifying the Final Layer . . . 7
7.5 Small CNN vs. ResNet-18 Comparison . . . 8
7.6 Data Size Recommendations . . . 8
7.7 Applications of Transfer Learning . . . 8

8 Checking PyTorch Setup **8**

9 Building a Small CNN Model **8**

10 Counting Model Parameters **9**

11 Summary **9**

1 Why Training a CNN from Scratch is Hard

Training from Scratch

Training a CNN from scratch means initializing all weights randomly and learning every parameter from the available data through forward and backward passes. While theoretically possible, this approach often becomes impractical for most real-world image tasks.

1.1 The Parameter Challenge

As a CNN learns, it develops many filters across multiple layers:

- **Early layers** learn edges, corners, and textures
- **Middle layers** learn shapes, repeated patterns, and object parts
- **Deep layers** learn class-specific structures and semantic patterns

As depth increases, the network becomes more powerful but also more expensive to train. Even though CNNs are more parameter-efficient than fully connected ANNs for images, modern CNNs still contain millions of trainable parameters.

Parameter Scale Examples:

- A small custom CNN: thousands to millions of parameters ($\sim$2 million)
- ResNet-18 (18 layers): $\sim$11.6 million parameters
- Large models: tens to hundreds of millions of parameters

1.2 Data Requirement Problem

A model with many parameters needs enough data to learn stable patterns. With insufficient data:

- The model may **memorize** training images (overfitting)
- Validation accuracy may remain poor
- The model may fail on new images

For image tasks, collecting labeled data is expensive:

- Images must be gathered manually
- Labels must be correct
- Classes must be balanced
- Data must capture real-world variation (pose, lighting, background, camera angle, partial visibility, breed differences)

A CNN trained from scratch must learn all these variations from the available dataset alone.

1.3 Computational Cost

Training a deep CNN from scratch involves repeated forward and backward passes over many batches. This requires:

- GPU memory
- Significant training time
- Tuning of learning rate and regularization
- Debugging unstable training

If the dataset is small, the model may still not generalize well even after spending significant compute resources. The real question is not just whether the model can be trained, but whether it can be trained **well enough, fast enough, and with enough data**.

1.4 The Hidden Issue: Feature Learning Complexity

Even a simple two-class problem like cat vs. dog is not truly simple. The model must learn to detect:

- Edges and curves
- Fur textures
- Ears, eyes, face structure
- Pose variation and breed variation
- Background suppression

Feature Learning vs. Classification

Though the classification task may be simple (just two classes), the **feature learning** problem is still difficult. The model requires substantial visual intelligence to learn meaningful representations from scratch, regardless of how few output classes exist.

2 What is Transfer Learning?

Transfer Learning

Transfer learning means reusing the knowledge learned from one task to solve another related task. Instead of training a CNN from random weights, we start from a pre-trained model that has already learned useful visual features from a large image dataset.

2.1 Core Idea

Instead of starting from random weights (where initialization is random, then backpropagation optimizes weights), transfer learning starts from a model that has already learned useful visual patterns from a large image dataset.

Analogy: Transfer learning is like learning from parents. When you are young, your parents have seen a much larger world. Instead of learning everything on your own, you learn most things from their knowledge. The initial weights are "passed" to you. Then you fine-tune based on your own experience. Similarly, in transfer learning:

1. A pre-trained model provides initial weights (knowledge from large dataset)
2. You do not start from zero—you start with already-trained weights
3. You then train on your specific data to adapt the model to your task

2.2 Why Transfer Learning Works

Transfer learning works because:

- **Early visual features are universal**—edges, textures, and gradients appear in nearly all images
- **Pre-trained models learned from massive datasets**—they have already captured rich visual representations
- **Only task-specific mapping needs to be learned**—dramatically reducing data and training time requirements

A pre-trained model has already learned features like how to detect edges, textures, curves, object parts, and spatial hierarchies. These patterns are common across many image tasks. Edges learned from detecting cars are also useful for detecting dogs, and vice versa, because an edge is an edge regardless of context.

2.3 When Transfer Learning May Fail

Transfer learning may fail for very domain-specific tasks. If the target domain is significantly different from the images in the pre-training dataset (e.g., cancer detection from medical images), then:

- The pre-trained model's general features may not be sufficient
- Medical images are fundamentally different from everyday images
- Training from scratch on domain-specific data may be necessary

Domain Specificity

Transfer learning works best when the new task involves images similar to what the pre-trained model has seen. For highly specialized domains (medical imaging, microscopy, satellite imagery of rare phenomena), a custom model trained from scratch on domain-specific data may outperform a transfer learning approach.

3 ImageNet and Pre-trained Models

3.1 The ImageNet Dataset

ImageNet

ImageNet is a large-scale image database containing approximately **1.2 million images** across **1,000 object classes**. Each class contains roughly 1,000 images ($1000 \times 1000 \approx 10^6$). The dataset contains generalized, everyday images—cars, humans, vegetables, fruits, animals, natural scenes—the kinds of things commonly seen in daily life.

ImageNet does **not** contain highly domain-specific images such as cancer scans or specialized medical data. It is a generalized dataset of common visual objects.

3.2 Famous Pre-trained CNN Architectures

Several well-known CNN architectures have been trained on ImageNet and are available as pre-trained models:

Architecture	Key Characteristic
ResNet	Residual connections, available in 18/34/50/101/152 layer variants
VGG	Simple architecture with small 3×3 filters
AlexNet	Pioneer deep CNN, won ImageNet 2012
EfficientNet	Balanced scaling of depth, width, and resolution
MobileNet	Lightweight, designed for mobile and edge devices
GoogLeNet (Inception)	Inception modules with parallel filter sizes

These pre-trained models already contain rich visual representations that can be reused for new tasks.

4 CNN Feature Hierarchy

Feature learning in CNN is **hierarchical**. Different layers learn different levels of abstraction:

CNN Feature Hierarchy:

1. **Early layers** — Learn general features: edges, gradients, simple textures
2. **Middle layers** — Learn combinations: shapes, object parts, repeated structures
3. **Deep layers** — Learn task-specific concepts: faces, wheels, animals, domain-specific textures

This hierarchy is extremely useful for transfer learning. Even if the new task is different from the original training task, **early visual features are still valuable**. For example, edges learned from detecting cars are also useful for detecting dogs because fundamental visual patterns are shared across all natural images.

Key Insight

All images—flowers, animals, vehicles—share similar low-level features: edges, textures, colors, backgrounds. A pre-trained model that has learned to detect these features can transfer this knowledge to virtually any image classification task.

5 Two Strategies in Transfer Learning

There are two common approaches to applying transfer learning:

5.1 Strategy 1: Feature Extraction

In feature extraction, the pre-trained CNN's convolutional layers are kept **completely frozen** (weights are fixed). Only a new classifier head is trained on top.

Steps:

1. Load a pre-trained model (e.g., ResNet-18 with ImageNet weights)
2. Freeze all convolutional layers (set `requires_grad = False`)
3. Replace the final classification layer to match the number of target classes
4. Train only the new classifier layer on the new dataset

The pre-trained CNN acts as a **feature extractor**—it converts input images into meaningful feature vectors. These vectors are then fed to the classifier (fully connected layers) for the final classification decision.

Feature Extraction and Encoders

The convolutional layers in feature extraction mode act similarly to an **encoder**. The key insight is that the "game" of deep learning is about extracting features and converting them into vectors. Once data is in vector/embedding form, any classifier (SVM, logistic regression, neural network) can be used on top. This same principle applies in NLP with word embeddings and transformers.

5.2 Strategy 2: Fine-Tuning

In fine-tuning, instead of freezing all layers, some deeper layers are allowed to update during training. This allows the model to slightly adapt its learned representations to the new dataset.

Steps:

1. Load a pre-trained model
2. Replace the final classification layer
3. Train the classifier first (with frozen backbone)
4. **Unfreeze some deeper layers**
5. Continue training with a **small learning rate**

Fine-tuning allows the model to adapt task-specific features in the deeper layers while preserving the general features learned in the early layers.

5.3 Comparison: Feature Extraction vs. Fine-Tuning

Aspect	Feature Extraction	Fine-Tuning
Frozen layers	All convolutional layers frozen	Only early/middle layers frozen
Trainable layers	Only classifier (FC layers)	Classifier + some deeper conv layers
Training speed	Faster (fewer parameters to update)	Slower (more parameters to update)
Data requirement	Works with smaller datasets	Benefits from more data
Adaptability	Uses generic features as-is	Adapts features to new domain
Use case	New task similar to original	New task somewhat different

6 Freezing Layers in PyTorch

Freezing a Layer

Freezing a layer means preventing its weights from updating during training. In PyTorch, this is done by setting `param.requires_grad = False` for each parameter in the layer.

When layers are frozen:

- Gradients are **not computed** for those layers
- Parameters **remain fixed** at their pre-trained values
- Training becomes **faster** because fewer parameters must be optimized

Freezing All Parameters

```python
import torch
import torchvision.models as models

# Load pre-trained ResNet-18
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
model = model.to(device)

# Freeze all parameters
for param in model.parameters():
    param.requires_grad = False

# After freezing: trainable parameters = 0
```

Listing 1: Freezing pre-trained model parameters

After freezing all parameters, the trainable parameter count drops to zero. The total parameters remain the same (11.6 million for ResNet-18), but none of them will be updated during training.

7 Practical Transfer Learning Workflow

7.1 Step-by-Step Pipeline

The typical workflow for applying transfer learning:

1. **Load the pre-trained CNN** (e.g., ResNet-18 with ImageNet weights)
2. **Replace the final layer** to match the target number of classes
3. **Freeze most layers** (keep convolutional backbone fixed)

4. **Train the classifier** on the new dataset
5. **Optionally fine-tune deeper layers** with a small learning rate

7.2 Loading Models: With and Without Pre-trained Weights

Loading ResNet-18

```python
import torchvision.models as models

# Without pre-trained weights (random initialization)
model_scratch = models.resnet18(weights=None).to(device)
# Total params = 11.6M, Trainable = 11.6M (all trainable)

# With pre-trained ImageNet weights
model_pretrained = models.resnet18(
    weights=models.ResNet18_Weights.DEFAULT
).to(device)
# Total params = 11.6M, Trainable = 11.6M (before freezing)
# Note: weights are downloaded from PyTorch model hub
```

Listing 2: Loading ResNet-18 with and without weights

7.3 Inspecting the Final Layer

ResNet-18 by default outputs 1,000 classes (matching ImageNet's 1,000 object categories). The final fully connected layer has:

- **In features:** 512 (the output of the last convolutional block)
- **Out features:** 1,000 (ImageNet classes)

For a custom task (e.g., cat vs. dog with 2 classes), this final layer must be replaced.

7.4 Modifying the Final Layer

Replacing the Classifier

```python
import torch.nn as nn

# Inspect original final layer
print(model_pretrained.fc)
# Linear(in_features=512, out_features=1000, bias=True)

# Replace with custom classifier for 2 classes
model_pretrained.fc = nn.Linear(512, 2)

# Now: out_features = 2 (cat vs dog)
```

Listing 3: Replacing the final layer for custom classification

7.5 Small CNN vs. ResNet-18 Comparison

Property	Small Custom CNN	ResNet-18
Total parameters	∼2 million	∼11.6 million
Depth	Few layers (3–5 conv layers)	18 layers
Training from scratch	Feasible but limited	Expensive, needs large data
With transfer learning	Not applicable (custom)	Highly effective
Output classes (default)	Task-specific	1,000 (ImageNet)

7.6 Data Size Recommendations

For transfer learning with pre-trained models:

- Datasets with >2,000 images generally work fine
- Recommended minimum: **at least 5,000 images**
- Datasets with <1,000 images risk overfitting even in the fully connected layers
- The pre-trained model has been trained on 1.2 million images, so even 5,000–10,000 is considered "small" in comparison

7.7 Applications of Transfer Learning

Transfer learning is used across many domains:

- Medical imaging
- Autonomous driving
- Defect detection (manufacturing)
- Satellite imagery analysis
- Wildlife monitoring
- Product recognition

8 Checking PyTorch Setup

Before coding in PyTorch, always verify the setup:

PyTorch Setup Verification

```
import torch

print(f"PyTorch version: {torch.__version__}")
print(f"CUDA available: {torch.cuda.is_available()}")

device = torch.device("cuda" if torch.cuda.is_available()
                      else "cpu")
print(f"Using device: {device}")
```

Listing 4: Verifying PyTorch and CUDA availability

9 Building a Small CNN Model

To understand the parameter difference, consider building a small custom CNN:

Small CNN Architecture

```python
import torch.nn as nn

class SmallCNN(nn.Module):
    def __init__(self, num_classes=2):
        super(SmallCNN, self).__init__()
        # Conv1: 3 -> 16 channels, 128x128
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        # Conv2: 16 -> 32 channels
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        # Conv3: 32 -> 64 channels
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        # Fully connected layers
        self.fc1 = nn.Linear(64 * 16 * 16, 120)
        self.fc2 = nn.Linear(120, num_classes)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))  # 128->64
        x = self.pool(torch.relu(self.conv2(x)))  # 64->32
        x = self.pool(torch.relu(self.conv3(x)))  # 32->16
        x = x.view(x.size(0), -1)  # Flatten
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# Input shape: (batch=4, channels=3, H=128, W=128)
# Total parameters: ~2 million
```

Listing 5: Small custom CNN for comparison

The small CNN already has a significant number of parameters (∼2 million). ResNet-18 is around 5 times larger. If all these parameters start randomly, the model needs enough data and compute to learn meaningful values—which is precisely why pre-trained weights are valuable.

10 Counting Model Parameters

Parameter Counting Utility

```python
def count_parameters(model):
    total = sum(p.numel() for p in model.parameters())
    trainable = sum(p.numel() for p in model.parameters()
                    if p.requires_grad)
    return total, trainable

# For a model trained from scratch:
# total == trainable (all parameters are trainable)

# After freezing:
# total remains the same, trainable drops to 0
# (or to the number of parameters in unfrozen layers)
```

Listing 6: Counting total and trainable parameters

11 Summary

- Training a CNN from scratch requires millions of parameters, large labeled datasets, and significant computational resources

- **Transfer learning** reuses knowledge from a pre-trained model, allowing effective training with smaller datasets and less compute
- **ImageNet** is the standard pre-training dataset: 1.2 million images across 1,000 classes of everyday objects
- Key pre-trained architectures: ResNet, VGG, AlexNet, EfficientNet, MobileNet, GoogLeNet
- CNN features form a **hierarchy**: early layers (edges, textures) → middle layers (shapes, parts) → deep layers (task-specific)
- **Feature extraction**: freeze all convolutional layers, train only the classifier
- **Fine-tuning**: unfreeze some deeper layers and train with a small learning rate
- Freezing in PyTorch: `param.requires_grad = False`
- Transfer learning works because early visual features are universal across image types
- Recommended minimum dataset size for transfer learning: 5,000+ images
- Transfer learning may fail for highly domain-specific tasks (e.g., medical imaging) where pre-trained features are insufficient